

Taller 6 – Principios S.O.L.I.D.

[Tu nombre]

21 de abril de 2026

1. Introducción

Este taller tiene como objetivo refactorizar una calculadora monolítica en Java aplicando principios **S.O.L.I.D.**, junto con prácticas de **TDD** y **DDD**.

2. Código Inicial (Fase 1)

2.1. Descripción

Se generó una clase única que contiene:

- Operaciones matemáticas
- Manejo de consola
- Validaciones
- Control de flujo

2.2. Problemas detectados

- Muchas responsabilidades en una sola clase
- Difícil de mantener
- Difícil de escalar
- No es testeable

3. Reflexión Crítica (Fase 2)

Las respuestas están escritas como commit en GitHub

4. Refactorización S.O.L.I.D. (Fase 3)

Cada una de las fases se ha hecho individualmente a medida de que se explicaba en la clase; y con un prompt para que antigravity hiciera el código

5. Interacción con IA (Antigravity)

5.1. Contexto Inicial y Objetivo del Proyecto

El proyecto inició con el requerimiento de construir una calculadora de consola en Java que soportara operaciones matemáticas binarias (suma, resta, multiplicación, división) y unarias (raíz cuadrada, logaritmo natural).

El problema base: Se comenzó partiendo de una **Clase Monolítica** en la que coexistían múltiples responsabilidades. En la misma clase se manejaba la impresión de menús, la lectura de datos mediante el escáner, el control de errores, y la lógica algorítmica matemática subyacente.

El objetivo principal fue realizar una refactorización incremental orientada por **Dominio (DDD)** y **Desarrollo Guiado por Pruebas (TDD)**, culminando en un programa que obedeciera estrictamente los cinco principios **SOLID** del diseño orientado a objetos.

5.2. Metodología: TDD (Test-Driven Development)

Durante todo el progreso de la refactorización, el proyecto se construyó usando TDD bajo el marco de pruebas de **JUnit**.

El flujo de trabajo constó de:

1. **Fase Roja:** Proponer escenarios y aserciones matemáticamente correctos para una clase o método que aún no existía.
2. **Fase Verde:** Codificar e implementar la mínima lógica estructural necesaria para solventar esas pruebas unitarias.
3. **Refactorización:** Acoplar la implementación exitosa de vuelta en el núcleo del sistema preservando la mantenibilidad.

5.3. Aplicación Evolutiva de los Principios SOLID

5.3.1. S - Single Responsibility Principle (SRP - Responsabilidad Única)

El inconveniente: La calculadora operaba I/O e insertaba reglas de negocio matemáticas a la vez. Cualquier ajuste mínimo a los textos de la consola arriesgaba dañar una fórmula.

La solución: Se desglosó el monolito inicial para crear un ecosistema modular:

- `ValidadorEntrada.java`: Tomó control exclusivo de interactuar con el escáner y la tolerancia a fallos.

- `OperacionesBinarias` y `OperacionesUnarias`: Responsables de los algoritmos matemáticos.

Evidencia del principio: Cada clase posee una única razón para cambiar.

5.3.2. O - Open/Closed Principle (OCP - Abierto/Cerrado)

El inconveniente: Agregar una operación como `Potencia` requería modificar una estructura `switch-case` existente.

La solución:

- Se creó la interfaz `Operacion`.
- Se implementaron clases como `Suma.java`, `Resta.java`, `Potencia.java`.
- Se utilizó un `HashMap` como registro de operaciones (polimorfismo dinámico).

Evidencia del principio: El sistema quedó abierto a extensión pero cerrado a modificación.

5.3.3. L - Liskov Substitution Principle (LSP - Sustitución de Liskov)

El inconveniente: Diferentes operaciones requerían distinta cantidad de operandos, causando errores en ejecución.

La solución:

- Se definió el método `getNumeroOperandos()`.
- Se añadieron validaciones con `IllegalArgumentException`.

Evidencia del principio: Todas las implementaciones pueden sustituirse sin afectar el sistema.

5.3.4. I - Interface Segregation Principle (ISP - Segregación de Interfaces)

El inconveniente: Una interfaz obligaba a manejar datos innecesarios (varargs).

La solución:

- Se crearon interfaces específicas:
 - `OperacionBinaria`
 - `OperacionUnaria`

Evidencia del principio: Ninguna clase depende de métodos innecesarios.

5.3.5. D - Dependency Inversion Principle (DIP - Inversión de Dependencias)

El inconveniente: El Main instanciaba directamente clases como `new Suma()`.

La solución:

- Se crearon:
 - MotorCalculadora
 - CalculadoraConsola
- Se utilizó inyección de dependencias vía constructor.
- Calculadora.java actúa como Composition Root.

Evidencia del principio: Las clases de alto nivel dependen de abstracciones, no de implementaciones concretas.